

OSSS-4

Question:

1. To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.

Solution:

```
ubuntu@ubuntu:~$ nano tokenizer.c
ubuntu@ubuntu:~$ gcc tokenizer.c -o tokenizer
ubuntu@ubuntu:~$ ./tokenizer
Enter Command: ls -l file.c

Tokens:
ls
-l
file.c

#include <stdio.h>
#include <string.h>

int main()
{
    char input[100];
    char *token;

    printf("Enter Command: ");
    fgets(input, sizeof(input), stdin);

    token = strtok(input, " \\t\\n");

    printf("\\nTokens:\\n");

    while(token != NULL)
    {
        printf("%s\\n", token);
        token = strtok(NULL, " \\t\\n");
    }

    return 0;
}

ubuntu@ubuntu:~$ nano parser.c
ubuntu@ubuntu:~$ gcc parser.c -o parser
ubuntu@ubuntu:~$ ./parser
Enter Command: grep main file.c
Argument 0 : grep
Argument 1 : main
Argument 2 : file.c

Syntax Valid
```

```

#include <stdio.h>
#include <string.h>

int main()
{
    char input[100];
    char *token;
    int count = 0;

    printf("Enter Command: ");
    fgets(input, sizeof(input), stdin);

    if(strcmp(input, "\n") == 0)
    {
        printf("Error: Empty Command\n");
        return 1;
    }

    token = strtok(input, " \t\n");

    while(token != NULL)
    {
        printf("Argument %d : %s\n",
            count,
            token);

        count++;

        token = strtok(NULL, " \t\n");
    }

    printf("\nSyntax Valid\n");

    return 0;
}

```

[Read 25.33 sec]

```

ubuntu@ubuntu:~$ ./parser
Enter Command: gcc file.c -o file
Argument 0 : gcc
Argument 1 : file.c
Argument 2 : -o
Argument 3 : file

Syntax Valid

```

```

ubuntu@ubuntu:~$ ./parser
Enter Command: grep main file.c
Argument 0 : grep
Argument 1 : main
Argument 2 : file.c

Syntax Valid

```